Theoretical Research

Programming Language II (JAVA)

Aizhan Esenbekova, COMFCI-24

What is Big-O time complexity?

Big O time complexity is basically a way of analyzing the run time (the amount of time it takes for our algorithm to execute as the input size of our algorithm grows)

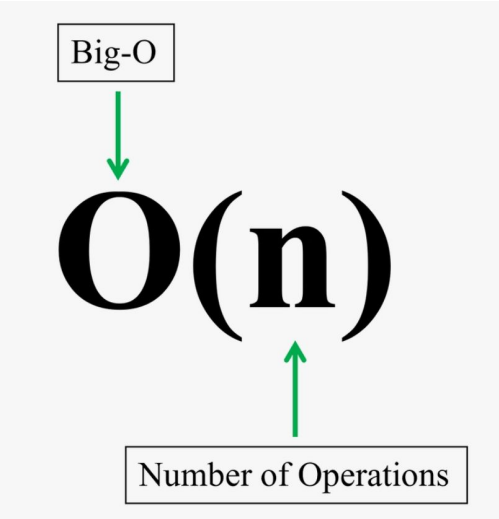
Удаляем константы:

$$O(\cancel{5}n) \rightarrow O(n)$$

$$O(n \cancel{+ 1}) \rightarrow O(n)$$

$$O(n^2 \cancel{+ n}) \rightarrow O(n^2)$$

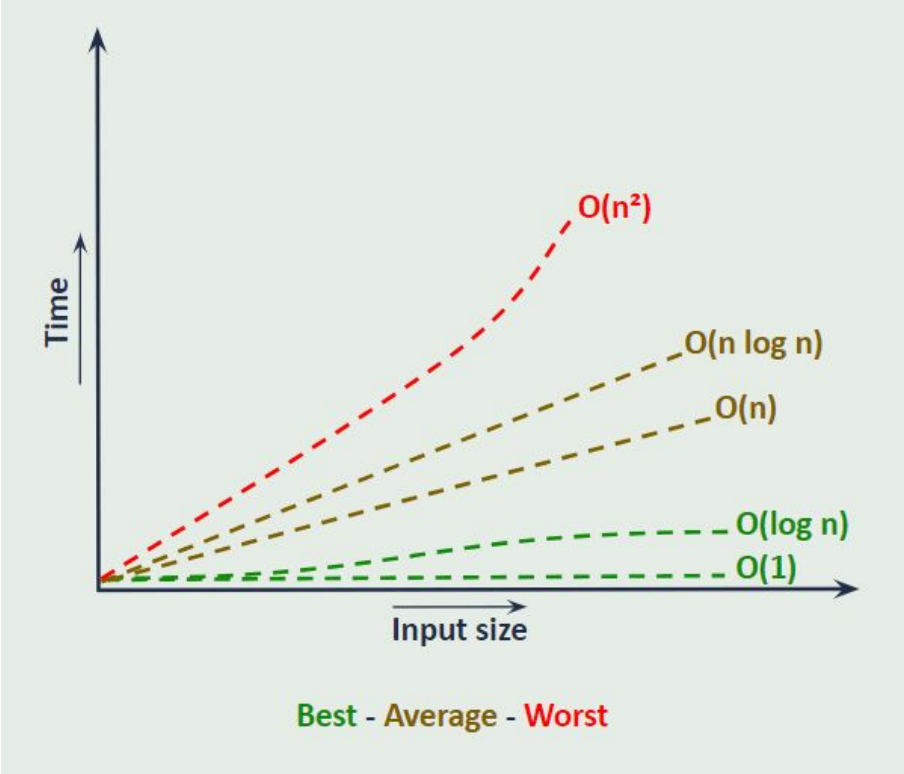
$$O(n + \cancel{\log n}) \rightarrow O(n)$$



Typically we can expect as the input what size of our algorithm grows the execution time of the algorithm is also going to grow (Y = X in terms of Big O this is represented as Big O of n **O(n)** where n is just a single variable that's our x-axis)

However, this is where we take a different turn from algebra you could have another function like this which is for example n divided by 2 (y = x/2) but we actually don't care about the differences when those differences are constant values we only care about the variable here which is n.

constants are removed.



Big O Notation: O(1) – Constant Time

Big O of 1 or (O(1)) is called constant time complexity.

This means that **no matter how large the input size becomes, the execution time stays the same.**

On a graph, this appears as **a flat horizontal line**, because the **time required does not change with the size of the input.** These are considered very efficient algorithms.

Array operations

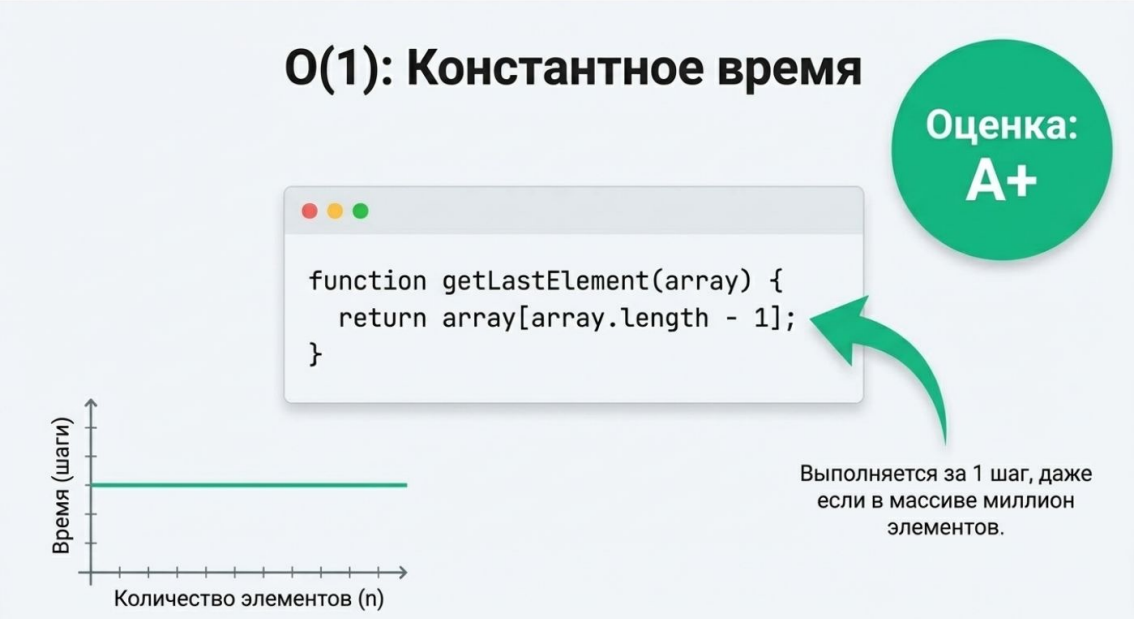
- Adding an element to the **end of an array**
- Removing an element from **the end of an array**
- Accessing an element **by index** (for example index 0, 1, or 2)

Hash Map / Hash Set operations

- **Insert** a key–value pair / **Insert** a value
- **Look up** a value by its key / **Check** if a value exists
- **Remove** a key–value pair / **Remove** a value

All of these operations typically run in **constant time O(1).**

```python
# Array
nums = [1, 2, 3]
nums.append(4)     # push to end
nums.pop()         # pop from end
nums[0]            # lookup
nums[1]
nums[2]

# HashMap / Set
hashMap = {}
hashMap["key"] = 10      # insert
print("key" in hashMap) # lookup
print(hashMap["key"])   # lookup
hashMap.pop("key")      # remove
```



Big-O Notation: O(log n) – Logarithmic Time

The problem size reduces by a constant factor each step (usually 2).

Logarithmic time complexity (O(log n)) means that the running time of an algorithm **increases slowly as the input size grows**. Instead of checking every element, the algorithm **reduces the problem size by a constant factor at each step**, usually by **dividing it in half**.

Because of this, even if the input size becomes very large, the number of steps needed increases only slightly.

A common example is binary search.

In binary search:

1. The algorithm looks at the **middle element** of a sorted array.
2. It decides whether the target value is in the **left half or the right half**.
3. Then it **repeats the process on the remaining half**.

Each step **cuts the search space in half**, which is why the complexity is **O(log n)**.

```java
public class LogarithmicTimeExample {  new *
    public static int binarySearch(int[] arr, int target){  1 usage  new *
        int left = 0, right = arr.length - 1;
        while(left <= right){
            int mid = (left + right)/2;
            if(arr[mid] == target) return mid;
            if(arr[mid] < target) left = mid + 1;
            else right = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args){  new *
        int[] arr = {1,2,3,4,5,6,7,8};
        System.out.println("Index: " + binarySearch(arr, target: 6));
```

Index: 5

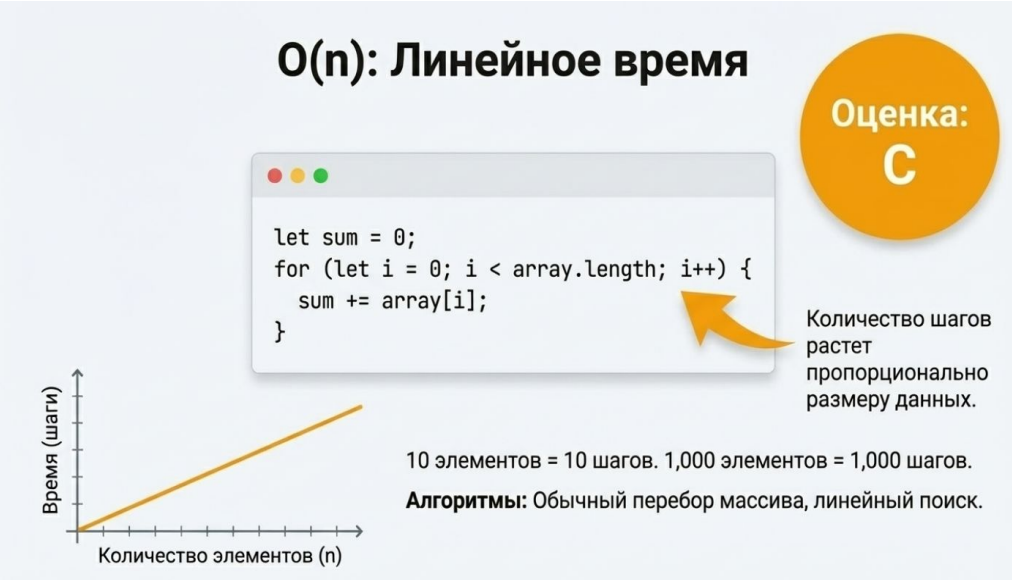
Big O Notation: O(n) – Linear Time

As the input size increases, the running time increases proportionally.

For example, if we have an array of values and we want to calculate the sum of all numbers, **we must go through every element in the array.**

- If the array has **3 values**, we perform 3 operations.
- If the array has **1,000,000 values,** we perform 1,000,000 operations.

Because the **algorithm must loop through all elements,** the running time grows with the size of the input.



Example: Sum of an Array (Java)

```java
public static void main(String[] args) {  new *

    // example with numbers
    int[] numbers = {1, 2, 3, 4, 5};

    // we call the sum function
    int result = sum(numbers);

    // Print the result
    System.out.println("Sum of array elements: " + result);
}
}
```

```java
public class SumArrayExample {  new *

    // Function to calculate the sum of array elements
    public static int sum(int[] arr) {  1 usage  new *
        int total = 0;

        for (int i = 0; i < arr.length; i++) {
            total += arr[i];
        }

        return total;
    }
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2
Sum of array elements: 15

Process finished with exit code 0
```

Time Complexity: O(n)

Because the loop goes through every element in the array.

Big O Notation: O(n) – Linear Time

Java Example: Linear Search

```java
public static int linearSearch(int[] arr, int target) {

    for (int i = 0; i < arr.length; i++) {

        if (arr[i] == target) {
            return i;
        }

    }

    return -1;
}
```

```java
public static void main(String[] args) {  new*

    // example with numbers
    int[] numbers = {1, 2, 3, 4, 5};
    int target = 5;

    int result = linearSearch(numbers, target);

    // Print the result
    System.out.println("Index of our target: " + result);
}
```

```
C:\Users\aizha\.jdks\openjd
Sum of array elements: 4
```

Time Complexity: O(n)

Searching an Array

Searching for a value in an unsorted array is also O(n).

In the worst case, the algorithm **must check every element before finding the value or determining that it does not exist.**

Example:
Searching for 100 in: [1, 2, 3, 4, 5]

The algorithm must check each element before concluding that 100 is not present.

```java
public static void main(String[] args) {  new*

    // example with numbers
    int[] numbers = {1, 2, 3, 4, 5};
    int target = 100;

    int result = linearSearch(numbers, target);

    // Print the result
    System.out.println("Index of our target: " + result);
}
}
```

```
C:\Users\aizha\.jdks\ope
Index of our target: -1

Process finished with ex
```

The algorithm must **check each element** before concluding that **100 is not present.**

Big O Notation: O(n) – Linear Time

Inserting in the Middle of an Array

In the worst case, **we may have to move every element in the array.**

Because Big O measures the worst-case scenario, inserting in the middle is O(n)

Example

Array: [1, 2, 3]

Insert 100 in the middle:

[1, 100, 2, 3]

To do this: Move 2 to the right + Move 3 to the right

```java
import java.util.ArrayList;

public class ArrayListInsertExample { new *

    public static void main(String[] args) { new *

        ArrayList<Integer> list = new ArrayList<>();

        // Add elements to the list
        list.add(1);
        list.add(2);
        list.add(3);

        // Print original list
        System.out.println("Original List: " + list);

        // Insert 100 at index 1
        list.add( index: 1,  element: 100);

        // Print updated list
        System.out.println("Updated List after insertion: " + list);
    }
}
```

```java
import java.util.ArrayList;

public class ArrayListRemoveExample { new *

    public static void main(String[] args) { new *

        // Create an ArrayList
        ArrayList<Integer> list = new ArrayList<>();

        // Add elements
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);

        // Print original list
        System.out.println("Original List: " + list);

        // Remove element at index 1
        list.remove( index: 1);

        // Print updated list
        System.out.println("Updated List after removal: " + list);
    }
}
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
Original List: [1, 2, 3]
Updated List after insertion: [1, 100, 2, 3]
```

Removing from the Middle of an Array

Removing from the middle is also O(n).

Original array:

[1, 2, 3, 4]

Remove element at index 0:

[2, 3, 4]

All elements must shift left.

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\
Original List: [1, 2, 3, 4]
Updated List after removal: [1, 3, 4]
```

Big O Notation: O(n) – Linear Time

Sometimes an algorithm looks like it has nested loops, so people assume the time complexity is $O(n^2)$. However, this is not always correct.

In some algorithms (like Sliding Window and Monotonic Stack), the inner loop does not run fully for every iteration of the outer loop. Instead, each element is processed a limited number of times, which keeps the overall complexity O(n).e size of the input.

1. Sliding Window Algorithm (O(n))

Problem Example: Find the maximum sum of any subarray of size k.

Why **this is O(n)** (not $O(n^2)$)?

Even though it may look like multiple operations are happening:

- Each element enters the window once
- Each element leaves the window once

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
Maximum sum of subarray of size 3 is: 9
```

```java
public class SlidingWindowExample {  new *

    public static int maxSumSubarray(int[] arr, int k) {  1 usage  new *

        int windowSum = 0;
        int maxSum = 0;

        // First window
        for (int i = 0; i < k; i++) {
            windowSum += arr[i];
        }

        maxSum = windowSum;

        // Slide the window
        for (int i = k; i < arr.length; i++) {

            windowSum += arr[i];       // add next element
            windowSum -= arr[i - k];   // remove first element of window

            maxSum = Math.max(maxSum, windowSum);
        }

        return maxSum;
    }
```

[2, 1, 5, 1, 3, 2]

Window size = 3

Windows move like this:

[2 1 5] 1 3 2
2 [1 5 1] 3 2
2 1 [5 1 3] 2
2 1 5 [1 3 2]

The window slides forward only once across the array meaning each element is processed a constant number of times.

So the total operations are proportional to n.

Therefore: **Time Complexity = O(n)**

Big O Notation: O(n^2) – Quadratic Time

Big O of n^2 (O(n^2)) occurs when the number of operations grows proportional to the square of the input size.

This usually happens when we **use nested loops** that iterate through the same dataset.

Example 1: Iterating a 2D Array

If we have a 2D grid (matrix) with size n × n,

we need two loops:

- One loop for **rows**
- One loop for **columns**

Time Complexity

Outer loop → **n**
Inner loop → **n**

Total operations:

$n \times n = n^2$

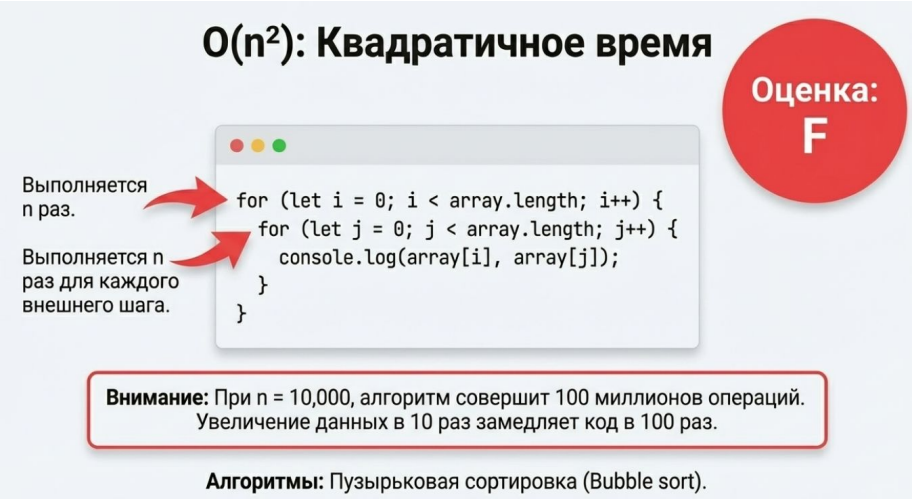
Therefore:

Time Complexity = O(n^2)

```java
public class MatrixExample {  new *

    public static void main(String[] args) {  new *

        int[][] grid = {
                {1,2,3},
                {4,5,6},
                {7,8,9}
        };

        for(int i = 0; i < grid.length; i++) {

            for(int j = 0; j < grid[i].length; j++) {

                System.out.print(grid[i][j] + " ");
```

Example 2: Iterating Through an Array Multiple Times

```java
public class NestedLoopExample {  new *

    public static void main(String[] args) {  new *

        int[] arr = {1,2,3,4};

        for(int i = 0; i < arr.length; i++) {

            for(int j = 0; j < arr.length; j++) {

                System.out.println(arr[i] + " " + arr[j]);
```



Big-O Notation: O(n × m) – Rectangular Matrix

O(n × m) is the time complexity when you have a data structure that has two dimensions with different sizes:

n = number of **rows** m = number of **columns**

This is common for rectangular matrices, tables, or grids where the height and width are not equal.

The total number of operations depends on both dimensions, because you usually **need to visit every element once.**

Comparison With O(n²)

- **O(n²)** is a **special case** of O(n × m) where n = m (square matrix).
- Example: 5 × 5 → O(25) → O(n²).
- Rectangular matrices keep both dimensions separate, because **rows and columns may differ significantly**, so the complexity is **not necessarily quadratic**.

Space Complexity

- Usually **O(1)** if you just traverse and process elements.
- Could be **O(n × m)** if you store intermediate results in another 2D array.

```java
public class RectangularMatrixExample {  new *

    public static void main(String[] args) {  new *

        // Rectangular matrix: 2 rows x 3 columns
        int[][] matrix = {
                {1, 2, 3},
                {4, 5, 6}
        };

        // Loop through each row
        for (int i = 0; i < matrix.length; i++) {
            // Loop through each column in the current row
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println(); // New line after each row
```

Key Points

```
1 2 3
4 5 6
```

1. **Outer loop runs n times**
   - Usually iterates over **rows**.
2. **Inner loop runs m times**
   - Usually iterates over **columns in that row**.
3. **Total operations = n × m**
   - Every element is processed exactly **once**.

Big O Notation

Big O notation describes the **upper bound** of an algorithm's running time. This means it represents the **worst-case scenario**—the maximum time an algorithm may take to complete as the input size increases.

In other words, Big O tells us that the algorithm **will not grow faster than a certain rate**. It gives a guarantee that the running time will stay **at or below a particular growth level**, even in the most unfavorable situation.

For example, if an algorithm has a time complexity of **O(n)**, it means that the number of operations grows **linearly with the size of the input**. If the input size doubles, the running time will approximately double as well.

Big O notation is very important because it helps programmers **predict the scalability of algorithms** and determine whether a solution will remain efficient when handling large datasets.

Summary

- **Big O (O)** → describes the **worst-case time complexity**.
- It shows the **maximum running time** an algorithm might require.
- It helps evaluate how well an algorithm will perform **as the input size grows**.

Example: Consider a function that finds the maximum element in an array.

The function iterates through the array once, making the time complexity O(n), where n is the number of elements in the array.

```
C:\Users\aizha\.jdks\
Maximum value: 9
```

```java
public class Main {
    public static void main(String[] args) {

        int[] arr = {3, 7, 2, 9, 5};

        int maxVal = arr[0];

        for (int num : arr) {
            if (num > maxVal) {
                maxVal = num;
            }
        }

        System.out.println("Maximum value: " + maxVal);
```

Big Theta Notation (Θ)

Big Theta (Θ) notation describes the **exact asymptotic running time of an algorithm**. Unlike Big O, which only shows the upper limit, Big Theta provides a **tight bound**, meaning it shows **both the upper and lower bounds of the algorithm's growth rate**.

This means that the algorithm's running time **grows at a specific rate and stays within that rate as the input size increases**. In other words, Big Theta indicates that the algorithm will **consistently perform within the same complexity level**, regardless of whether the situation is best, average, or worst case (within the same growth order).

Big Theta is useful because it gives a **more precise description of an algorithm's performance**. It tells us that the running time is **neither faster nor slower than a particular growth rate**, but instead follows that rate closely as the input size increases.

Why it is Θ(log n)

- Binary search works on a **sorted array**.
- In each iteration, the algorithm **divides the search range in half**.
- Because the number of elements being checked **reduces by half each time**, the number of steps grows logarithmically.
- Therefore, the running time is **Θ(log n)**.

```java
public static int binarySearch(int[] arr, int target) {  1 usage

    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {

        int mid = (left + right) / 2;

        if (arr[mid] == target) {
            return mid;
        }
        else if (arr[mid] < target) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
```

```java
public static void main(String[] args) {

    int[] arr = {1, 3, 5, 7, 9, 11, 13};
    int target = 7;

    int result = binarySearch(arr, target);

    if (result != -1) {
        System.out.println("Element found at index: " + result);
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\b
Element found at index: 3

Process finished with exit code 0
```

Big Omega (Ω)

Big Omega (Ω) notation describes the **lower bound of an algorithm's running time**. It represents the **best-case scenario**, meaning the minimum amount of time an algorithm needs to complete its task.

In other words, Big Omega shows that the algorithm **cannot run faster than a certain limit**, even in the most favorable situation. It tells us the **least number of operations** that must be performed for the algorithm to finish.

Big Omega is useful because it helps us understand the **best possible performance** of an algorithm. While Big O focuses on the worst case and Big Theta describes the exact growth rate, Big Omega highlights the **minimum time required when everything works in the most efficient way**.

Explanation:

In the **best case**, the target value is located **in the first position of the array**. The algorithm only needs **one comparison** to determine the result.

Because the algorithm finishes after a **constant number of operations**, regardless of the array size, the best-case time complexity is **Ω(1)** (constant time).

Example: Consider a simple function that **checks whether the first element of an array is equal to a target value**.

```java
public static boolean isFirstElementTarget(int[] arr, int target) {  1 usage

    if (arr[0] == target) {
        return true;
    }

    return false;
}

public static void main(String[] args) {

    int[] arr = {5, 10, 15, 20};
    int target = 5;

    boolean result = isFirstElementTarget(arr, target);

    System.out.println("Is first element the target? " + result);
}
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\
Is first element the target? true

Process finished with exit code 0
```

Comparing the Notations

Purpose and Usage

- **Big O (O)** is used to describe the **worst-case running time** of an algorithm. It shows the maximum amount of time the algorithm may take when the input size increases. This helps developers understand how slow an algorithm could become in the most demanding situations.

- **Big Theta (Θ)** provides a **tight bound** for an algorithm's running time. It describes both the upper and lower limits of the algorithm's growth rate, meaning it shows the **exact order of growth** when the algorithm behaves consistently.

- **Big Omega (Ω)** represents the **best-case running time**. It shows the minimum time an algorithm needs to finish when everything happens in the most favorable way.

Relationship Between the Notations

These three notations are related because they all describe how the **running time grows as the input size increases**, but they focus on different limits.

- **Big O** focuses on the **upper bound**, meaning the algorithm will **not take more time than a certain limit**.

- **Big Theta** describes **both the upper and lower bounds**, showing that the algorithm's running time grows at a **specific and predictable rate**.

- **Big Omega** focuses on the **lower bound**, meaning the algorithm will take **at least a certain amount of time**, even in the best possible case.

Worst-Case Analysis (Big O)

Worst-case analysis describes the **maximum amount of time an algorithm might take** for any input of size *n*. It focuses on the situation where the algorithm performs the **largest number of operations possible**. This is why worst-case complexity is usually expressed using **Big O notation**.

Big O is widely used because it provides a **guarantee about performance**. It tells us that the algorithm **will not take more time than a certain limit**, even in the most difficult or unfavorable inputs. For example, if an algorithm has a time complexity of **O(n²)**, it means that in the worst case the number of operations will grow proportionally to the square of the input size.

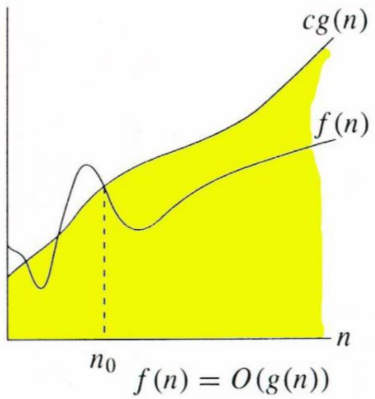
Worst-case analysis is especially important in **systems where reliability and predictable performance are required**, such as databases, real-time systems, or large-scale applications. Developers prefer this analysis because it prepares them for the **slowest possible scenario**, ensuring the algorithm still performs acceptably even under heavy workloads.

Example:
In a **linear search**, if the element we are looking for is **not in the array or is located at the last position**, the algorithm must check every element. If the array has *n* elements, it will perform *n* comparisons, resulting in a **worst-case time complexity of O(n)**.

O-notation

For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$O(g(n)) = \{f(n)$: there exist positive constants c and n_0 such that $0 \le f(n) \le cg(n)$, for all $n \ge n_0\}$



We say $g(n)$ is an *asymptotic upper bound* for $f(n)$

Big-O notation
(Upper Bound – Worst Case)

- Express the execution time as a function of the input size **n**
- Since only the growth rate matters, we can ignore the multiplicative constants and the lower order terms, e.g.,
  - $n, n+1, n+80, 40n, n+\log n$ is $O(n)$
  - $n^{1.1} + 10000000000n$ is $O(n^{1.1})$
  - n^2 is $O(n^2)$
  - $3n^2 + 6n + \log n + 24.5$ is $O(n^2)$
- $O(1) < O(\log n) < O((\log n)^3) < O(n) < O(n^2) < O(n^3) < O(n^{\log n}) < O(2^{\mathrm{sqrt}(n)}) < O(2^n) < O(n!) < O(n^n)$
- Constant < Logarithmic < Linear < Quadratic< Cubic < Polynomial < Factorial < Exponential

Average-Case Analysis (Theta)

Average-case analysis describes the **expected running time of an algorithm over all possible inputs** of size *n*. Instead of focusing only on the best or worst situations, it attempts to estimate how the algorithm performs **in typical conditions**. This is often represented using **Theta (Θ) notation** when the average behavior follows a specific growth rate.

To determine the average-case complexity, we must **consider every possible input, calculate the running time for each case, and then compute the average**. Because this requires understanding the **probability distribution of inputs**, it can be mathematically more difficult than worst-case analysis.

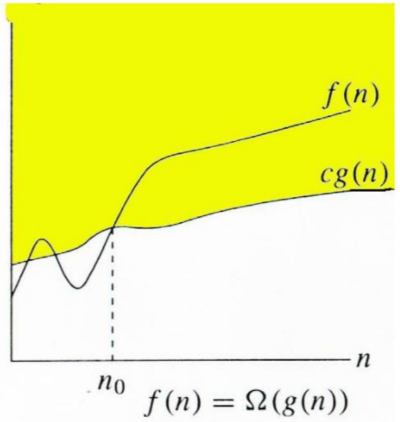
Average-case analysis can provide a **more realistic measure of performance**, since in real-world situations algorithms usually run on **random or varied data**, not only on the worst-case inputs.

Example:
Consider **linear search** again. If the element is equally likely to appear in any position in the array, on average the algorithm will check **about half of the elements** before finding it. Even though the average number of comparisons is roughly *n/2*, the complexity is still proportional to *n*, so the average-case complexity is **Θ(n)**.

Ω-notation

For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$\Omega(g(n)) = \{f(n)$: there exist positive constants c and n_0 such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0\}$



We say $g(n)$ is an *asymptotic lower bound* for $f(n)$

Ω-notation (Omega)
(Lower Bound – Best Case)

- We say Insertion Sort's run time T(n) is Ω(n)
- For example
  - the worst-case running time of insertion sort is $O(n^2)$, and
  - the best-case running time of insertion sort is Ω(n)
  - Running time falls anywhere between a linear function of **n** and a quadratic function of $\mathbf{n^2}$
- Example: $\sqrt{n} = \Omega(\lg n)$, with c = 1 and n0 = 16.

Worst-Case Analysis VS Average-Case Analysis

Aspect	Worst-Case Analysis (Big O)	Average-Case Analysis (Theta)
Definition	Describes the **maximum time** an algorithm may take for any input of size *n*.	Describes the **expected running time** of an algorithm over all possible inputs.
Focus	Focuses on the **slowest possible execution**.	Focuses on the **typical or average behavior**.
Notation Used	Usually expressed using **Big O (O)** notation.	Often described using **Theta (Θ)** when the average growth rate is known.
Difficulty of Calculation	**Easier to determine**, because we only analyze the most demanding case.	**More difficult to calculate**, because it requires analyzing all possible inputs and their probabilities.
Reliability	Provides a **guarantee that the algorithm will not perform worse than the given bound**.	Gives a **more realistic estimate of everyday performance**, but without worst-case guarantees.
Common Usage	**Most widely used in practice**, especially in software engineering and system design.	Used when researchers want a **more accurate representation of typical behavior**.

Worst-case analysis ensures that the algorithm will **never exceed a certain running time**, making it very useful for performance guarantees.

Average-case analysis tries to estimate **how the algorithm usually performs**, which can be more realistic but harder to compute.

Amortized Analysis

Amortized analysis is a method used in algorithm analysis to evaluate the **average cost of operations over a sequence of operations**. Instead of focusing on the cost of a single operation, it looks at how expensive and inexpensive operations are distributed across many steps.

In some algorithms, **most operations are very fast**, but occasionally a much **more expensive operation occurs**. If we only analyze that expensive operation using worst-case analysis, it may seem that the algorithm is inefficient. However, amortized analysis shows that when we **spread the cost of the expensive operations across many cheap operations**, the **average cost per operation remains small**.

This means that even though some individual operations may take longer, the **overall performance of the algorithm remains efficient when many operations are performed**.

Example: Dynamic Array (ArrayList)

A good example is a **dynamic array**, such as **Java's ArrayList**.

Normally, inserting a new element at the end of the array takes **O(1)** time. The element is simply placed in the next available position.

However, if the array becomes **full**, the program must:

1. Create a **new larger array** (usually double the size).
2. **Copy all existing elements** into the new array.
3. Insert the new element.

This resizing operation takes **O(n)** time because all elements must be copied. Even though this step is expensive, it **does not happen often**.

When we analyze many insert operations together, most of them take **O(1)** time, and only a few require resizing. Therefore, when we calculate the **average cost per insertion over many operations**, the time complexity becomes:

Amortized Time Complexity: O(1)

Complexity Table

Data Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
ArrayList	$O(1)$	$O(n)$	$O(1)$ amortized (end) / $O(n)$ middle	$O(n)$
HashSet	—	$O(1)$ average	$O(1)$ average	$O(1)$ average
HashMap	—	$O(1)$ average (by key)	$O(1)$ average	$O(1)$ average
ArrayDeque	$O(1)$ (front/back)	$O(n)$	$O(1)$ (front/back)	$O(1)$ (front/back)

- **O(1)** → **Constant time** → the operation always takes the same time, no matter how many elements there are.
- **O(n)** → **Linear time** → the time increases as the number of elements (**n**) increases.
- **n** → the **number of elements** in the data structure.
- **O(1) amortized** → the **average time is constant**, even though sometimes the operation may temporarily take longer.
- **O(1) average** → the operation usually takes **constant time on average** (for example in hash tables).
- **—** → the operation is **not applicable / not supported** for that data structure.
- **front/back** → the **beginning or the end** of the data structure (used in structures like **ArrayDeque**).

- **Arrays and ArrayLists** are efficient for **fast access operations**, since elements are stored in contiguous memory and can be accessed directly by index in **O(1)** time. However, inserting or deleting elements in the middle requires shifting other elements, which results in **O(n)** complexity.
- **HashSet and HashMap** are designed for **fast search, insertion, and deletion operations** using hashing. By computing a hash value, the structure can directly locate the storage position of elements or keys, allowing these operations to be performed in **O(1) average time**.
- **ArrayDeque** is optimized for **adding and removing elements from both ends of the structure**. Because it uses a circular array implementation, elements can be inserted or removed from the **front or back in constant time O(1)** without shifting other elements.
- Therefore, each data structure is chosen because its design makes it **efficient for the types of operations it commonly performs**.

What “front” and “back” mean

The terms **front** and **back** refer to the **two ends of a deque (double-ended queue)**.

A **deque** allows elements to be **added or removed from both sides**.

Example:

Front → [10, 20, 30, 40] ← Back

- **Front** → the **beginning** of the structure
- **Back** → the **end** of the structure

Operations:

Operation	Meaning
addFirst()	insert element at the **front**
addLast()	insert element at the **back**
removeFirst()	remove element from the **front**
removeLast()	remove element from the **back**

Example in Java:

```
import java.util.ArrayDeque;

ArrayDeque<Integer> deque = new ArrayDeque<>();

deque.addFirst(10);  // added to front
deque.addLast(20);  // added to back

deque.removeFirst(); // remove from front
deque.removeLast();  // remove from back
```

Because these operations **do not require shifting elements**, they take **O(1) time**.

Complexity Table

Data Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
ArrayList	$O(1)$	$O(n)$	$O(1)$ amortized (end) / $O(n)$ middle	$O(n)$
HashSet	—	$O(1)$ average	$O(1)$ average	$O(1)$ average
HashMap	—	$O(1)$ average (by key)	$O(1)$ average	$O(1)$ average
ArrayDeque	$O(1)$ (front/back)	$O(n)$	$O(1)$ (front/back)	$O(1)$ (front/back)

The empty spaces in the table mean that the **operation is not applicable or not typically defined for that data structure**.

For example:

- **HashSet** does not support **direct access by index**, because elements are stored using **hashing**, not in a specific order like arrays or lists. Therefore, an **Access operation does not exist**, so the table shows **—**.

- **HashMap** also does not support **index-based access**. Instead, elements are accessed using **keys**. That is why the table shows **—** under Access and specifies **O(1) average (by key)** under Search.

Array Example

```java
public class Main {
    public static void main(String[] args) {

        int n = 1000000;
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }

        // Access (O(1))
        long start = System.nanoTime();
        int value = arr[500000];
        long end = System.nanoTime();
        System.out.println("Array access time: " + (end - start));

        // Search (O(n))
        start = System.nanoTime();
        int target = n - 1;
        for (int i = 0; i < n; i++) {
            if (arr[i] == target) break;
        }
        end = System.nanoTime();
        System.out.println("Array search time: " + (end - start));
    }
}
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
Array access time: 3200
Array search time: 3289200
```

```java
int value = arr[500000];
```

Access – O(1)

Accessing an element in an array using its index takes **constant time**. This happens because arrays store elements in **contiguous memory locations**, allowing the program to directly calculate the memory address of any element using the index. The algorithm does **not need to search through other elements** to reach the required value. Therefore, regardless of how large the array becomes, the number of operations needed remains the same, which is why the complexity is **O(1)**.

```java
for (int i = 0; i < n; i++) {
    if (arr[i] == target) break;
}
```

Search – O(n)

In this case, the algorithm searches for a value by **checking each element sequentially**. If the target element is located at the **end of the array or is not present**, the loop must examine **all n elements**. Since the number of operations increases proportionally with the input size nnn, the running time grows linearly. Therefore, the time complexity of this search operation is **O(n)**.

ArrayList Example

```java
public class Main {
    public static void main(String[] args) {

        int n = 100000;
        ArrayList<Integer> list = new ArrayList<>();

        long start = System.nanoTime();

        for (int i = 0; i < n; i++) {
            list.add(i);        }

        long end = System.nanoTime();
        System.out.println("ArrayList insertion time: " + (end - start));

        start = System.nanoTime();
        list.contains(n - 1);
        end = System.nanoTime();

        System.out.println("ArrayList search time: " + (end - start));
    }
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
ArrayList insertion time: 6165900
ArrayList search time: 3543700
```

```java
list.add(i);
```

Insert – O(1) amortized

When inserting an element at the end of an ArrayList, the element is usually placed in the **next available position** in the internal array. This operation requires only **one step**, so it takes constant time. However, when the internal array becomes full, the ArrayList must **create a larger array and copy all existing elements** to the new array. This resizing operation takes **O(n)** time. Since resizing happens only occasionally while most insertions remain constant time, the **average cost per insertion over many operations is O(1)**. This is why it is called **amortized O(1)**.

```java
list.contains(n - 1);
```

Search – O(n)

The contains() method checks whether a value exists in the list by **iterating through the elements one by one**. If the element is located near the end of the list or is not present, the algorithm must **check all elements**. Because the number of comparisons grows linearly with the number of elements in the list, the time complexity of this operation is **O(n)**.

HashSet Example

```java
public class Main   no usages   1 inheritor
    public static void main(String[] args) {

        HashSet<Integer> set = new HashSet<>();
        int n = 100000;

        long start = System.nanoTime();

        for (int i = 0; i < n; i++) {
            set.add(i);
        }

        long end = System.nanoTime();
        System.out.println("HashSet insertion time: " + (end - start));

        start = System.nanoTime();
        set.contains(n - 1);
        end = System.nanoTime();

        System.out.println("HashSet search time: " + (end - start));
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
HashSet insertion time: 16480700
HashSet search time: 38100
```

```java
set.add(i);
set.contains(n - 1);
```

Insert / Search – O(1) average

HashSet uses a **hash table** to store elements. When inserting or searching for an element, the algorithm calculates a **hash value** that determines where the element should be stored. This allows the program to **directly access the correct location** instead of scanning all elements. Because of this direct access, both insertion and search operations usually take **constant time**, resulting in an average complexity of **O(1)**. In rare cases where many elements share the same hash value (called collisions), the performance can degrade, but on average it remains constant.

HashMap Example

```java
public class Main {
    public static void main(String[] args) {

        HashMap<Integer, String> map = new HashMap<>();
        int n = 100000;

        for (int i = 0; i < n; i++) {
            map.put(i, "Value" + i);
        }

        long start = System.nanoTime();
        map.get(n - 1);
        long end = System.nanoTime();

        System.out.println("HashMap access time: " + (end - start));
    }
}
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe '
HashMap access time: 38100
```

Access – O(1) average

HashMap also uses a **hash table structure**. Each key is converted into a **hash code**, which determines the bucket where the value is stored. When retrieving a value using get(), the algorithm computes the hash code and directly accesses the correct bucket. Since the number of operations does not depend on the size of the map in most cases, the access operation typically runs in **constant time**, giving an average time complexity of **O(1)**.

```java
map.get(n - 1);
```

ArrayDeque Example

```java
public class Main {
    public static void main(String[] args) {

        ArrayDeque<Integer> deque = new ArrayDeque<>();
        int n = 100000;

        long start = System.nanoTime();

        for (int i = 0; i < n; i++) {
            deque.addLast(i);
        }

        long end = System.nanoTime();
        System.out.println("ArrayDeque insertion time: " + (end - start));

        start = System.nanoTime();
        deque.removeFirst();
        end = System.nanoTime();
```

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\j
ArrayDeque insertion time: 6656700
ArrayDeque deletion time: 17000
```

Insert / Delete – O(1)

ArrayDeque is implemented using a **circular array**, which allows elements to be added or removed efficiently from both ends. When inserting at the end or removing from the front, the structure simply **updates pointers to the next position** without shifting other elements. Because the number of steps required remains constant regardless of the number of elements in the deque, these operations have **O(1)** time complexity.

```
deque.addLast(i);
deque.removeFirst();
```

System.NanoTime() mini test result

The results show that **array access (3200 ns)** is extremely fast compared to array search (3,289,200 ns). This confirms the theoretical complexity: array access is **O(1)** while searching requires checking elements sequentially, resulting in **O(n)** time.

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
Array access time: 3200
Array search time: 3289200
```

For **ArrayList**, insertion took **6,165,900 ns**, while searching took **3,543,700 ns**. This behavior aligns with theory: inserting at the end of an ArrayList is **amortized O(1)**, while searching requires scanning elements, giving **O(n)** complexity.

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\java.exe
ArrayList insertion time: 6165900
ArrayList search time: 3543700
```

The **HashSet** results show insertion at **16,480,700 ns** and search at **38,100 ns**. The search operation is extremely fast because HashSet uses hashing to directly locate elements, which corresponds to **O(1) average time**.

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\j
HashSet insertion time: 16480700
HashSet search time: 38100
```

For **HashMap**, accessing a value by key took **38,100 ns**, which also demonstrates **O(1) average complexity** due to the hash table structure.

```
C:\Users\aizha\.jdks\openjdk-25.0.2\
HashMap access time: 38100
```

Finally, **ArrayDeque** insertion took **6,656,700 ns**, while deletion took only **17,000 ns**. These operations are efficient because ArrayDeque allows constant-time insertion and removal from both ends, which matches the expected **O(1)** complexity.

```
C:\Users\aizha\.jdks\openjdk-25.0.2\bin\j
ArrayDeque insertion time: 6656700
ArrayDeque deletion time: 17000
```

Overall, the empirical results follow the **expected theoretical time complexities**, demonstrating how different data structures are optimized for different types of operations.